Содержание

ВВЕДЕНИЕ.....2

1 ОБЩАЯ ХАРАКТЕРИСТИКА ПРЕДПРИЯТИЯ.....3

1.1. Организационная структура предприятия.....3

1.2. Внутренний распорядок работы предприятия, охрана труда ИТ-специалистов.....4

1.3 Должностные инструкции, права и ответственность ИТ-специалистов в компании.....6

2 ОСУЩЕСТВЛЕНИЕ ИНТЕГРАЦИИ ПРОГРАММНЫХ МОДУЛЕЙ.....8

2.1 Разработка требований к программным модулям на основе анализа проектной и технической документации.....8

2.2 Выполнение интеграции модулей в программное обеспечение.....9

2.3 Выполнение отладки программного модуля с использованием специализированных программных средств.....10

2.4 Осуществление разработки тестовых наборов и тестовых сценариев 10

2.5 Инспектирование компонентов программного обеспечения на предмет соответствия стандартам кодирования.....11

3 ВЫПОЛНЕННЫЕ ЗАДАНИЯ.....13

ЗАКЛЮЧЕНИЕ.....17

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....18

ВВЕДЕНИЕ

Моя производственная практика прошла в филиале университета «Синергия» города Череповца с 17 по 30 ноября 2025 года. Практику курировал со стороны организации старший разработчик Мельников Олег Николаевич, а со стороны учебного заведения — педагог спец дисциплин Материкова Анжелика Аркадьевна.

Необходимость прохождения практики продиктована ускоренным переходом общества к цифровой экономике, особенно заметному в образовательной отрасли. Сегодня университеты, подобные «Синергии», становятся многоуровневыми цифровыми платформами, которым необходимы стабильное функционирование, высокая производительность и надёжность защиты данных. Работа специалиста сферы IT здесь выходит далеко за рамки простого программирования, охватывая весь процесс создания ПО: начиная от сбора требований и проектирования и заканчивая внедрением и сопровождением продукта. Именно умение интегрировать программные модули становится одним из важнейших навыков для разработчиков.

Основной целью моей производственной практики было закрепить и развить компетенции, приобретённые при освоении профессионального модуля ПМ.02 «Интеграция программных компонентов», непосредственно применяя их в условиях реальной IT-компании.

1 ОБЩАЯ ХАРАКТЕРИСТИКА ПРЕДПРИЯТИЯ

1.1. Организационная структура предприятия

«Синергия» входит в число наиболее крупных и активно развивающихся образовательных учреждений нашей страны. Череповецкий филиал вуза занимает важное положение в подготовке специалистов для приоритетных направлений экономики региона, включая металлургию, химпромышленность, сферу высоких технологий и бизнеса. Этот факт отражается в ориентации учебного заведения на подготовку выпускников с сильным практическим уклоном и глубокими контактами с индустриальным сектором.

Организация структуры череповецкого филиала построена по линейно-функциональному принципу, сочетающему четкую вертикаль власти и специализацию функций персонала. Главой филиала является Директорат, осуществляющий общий надзор и управление. Под непосредственным началом руководителя действуют заместители:

Заместитель по учебному процессу: контролирует образовательную деятельность, уровень преподавателей и поддерживает методическую базу.

Заместитель по хозяйственной части: обеспечивает нормальную эксплуатацию помещений и оборудование.

Заместитель по развитию: занимается перспективным планированием, привлечением партнеров и продвижением бренда.

Особое значение для целей производственной практики имеет ИТ-отдел. Его сотрудники одновременно выполняют административные обязанности перед директором филиала и технически подконтрольны центральному управлению ИТ московского кампуса, что гарантирует единый стандарт

информационной безопасности и регламентированную инфраструктуру на уровне всего холдинга.

Структурные элементы ИТ-отдела включают:

Руководитель подразделения: руководит общей работой, планирует ресурсы и координирует взаимодействие с коллегами и администрацией.

Системные администраторы (2 сотрудника): поддерживают работоспособность серверов, сетей, хранилищ данных, занимаются бэкапом и физической защитой оборудования.

Программисты-разработчики (3 человека): разрабатывают, модернизируют и обслуживают программное обеспечение филиала, создают учебные онлайн-платформы, интегрируют сервисы сторонних компаний.

Техподдержка (1 сотрудник): принимает первичные обращения пользователей, помогает решать повседневные проблемы и фиксирует запросы в специализированной системе учета заявок.

Подобная организация позволила эффективно разделить ответственность внутри коллектива и оперативно реагировать на возникающие потребности.

1.2. Внутренний распорядок работы предприятия, охрана труда ИТ-специалистов

Внутренний распорядок регламентирован Правилами внутреннего трудового распорядка филиала.

Рабочий график: 5-дневная рабочая неделя с понедельника по пятницу. Рабочий день с 9:00 до 18:00, обеденный перерыв с 13:00 до 14:00.

Ежедневные утренние планерки (9:15, 15 минут): проводятся начальником отдела для оперативного обсуждения текущих задач, статусов по проектам и возможных проблем.

Еженедельные планёрки (понедельник, 10:00): предназначены для подведения итогов прошлой недели, планирования работ на новую неделю и обсуждения стратегических вопросов.

Система управления проектами Jira: используется для постановки задач, отслеживания их выполнения, управления временем (time tracking) и ведения agile-досок (Scrum/Kanban).

Система документирования Confluence: служит единой базой знаний для хранения технической документации, регламентов, инструкций и отчетов.

Система контроля версий Git (GitLab): является основным инструментом для совместной разработки и управления исходным кодом.

Все сотрудники проходят вводный инструктаж по охране труда при приеме на работу и первичный инструктаж на рабочем месте.

Ежегодно проводится проверка знаний по охране труда.

Электробезопасность: Обязательный инструктаж с присвоением I группы по электробезопасности, так как работа ведется с офисной техникой, подключенной к электрической сети.

Гигиена труда: Строгое соблюдение регламентированных перерывов. При работе за компьютером более 2-х часов подряд предусмотрен перерыв продолжительностью 15 минут для снижения зрительного и мышечного напряжения.

Эргономика рабочих мест: Рабочие места оборудованы регулируемыми по высоте столами и креслами с ортопедическими спинками, подставками для ног. Мониторы имеют антибликовое покрытие и устанавливаются на расстоянии не менее 50 см от глаз.

Запреты: Запрещено самостоятельное вскрытие, ремонт или модификация любого оборудования без соответствующего допуска и уведомления системного администратора.

Пожарная безопасность: Рабочие места обеспечены огнетушителями (воздушно-пенными или порошковыми), сотрудники ознакомлены с путями эвакуации и действиями при пожаре.

1.3 Должностные инструкции, права и ответственность ИТ-специалистов в компании

В ходе практики были изучены должностные инструкции ключевых сотрудников IT-отдела:

1) Системный администратор:
    1. Обязанности: администрирование серверов (Windows/Linux), поддержка сети, установка и обновление ПО, резервное копирование, мониторинг инфраструктуры, техническая поддержка пользователей.
    2. Ответственность: за бесперебойную работу IT-инфраструктуры, целостность и конфиденциальность данных.
2) Разработчик (Программист):
    1. Обязанности: разработка и доработка ПО на C# и JavaScript, интеграция модулей через API, написание чистого кода, участие в создании техзаданий, исправление ошибок.
    2. Ответственность: за качество, своевременность выполнения задач и соответствие кода стандартам.
3) Специалист технической поддержки:

1. Обязанности: консультирование пользователей, регистрация и первичная обработка инцидентов в Service Desk, удаленное решение проблем, эскалация сложных задач.
2. Ответственность: за соблюдение SLA, вежливость и качество коммуникации.

4) Права IT-специалистов:

5) Запрашивать информацию, необходимую для выполнения обязанностей.
6) Вносить предложения по улучшению инфраструктуры и процессов.
7) Повышать квалификацию, в том числе за счет компании.
8) Требовать от коллег соблюдения правил IT-безопасности.

9) Ответственность: сотрудники несут дисциплинарную, материальную и в установленных случаях уголовную ответственность за неисполнение обязанностей, нарушение правил внутреннего распорядка, техники безопасности и разглашение конфиденциальной информации.

2 ОСУЩЕСТВЛЕНИЕ ИНТЕГРАЦИИ ПРОГРАММНЫХ МОДУЛЕЙ

2.1 Разработка требований к программным модулям на основе анализа проектной и технической документации

Была поставлена задача доработать модуль «Электронное портфолио студента», добавив функционал «Загрузка и верификация документов». В процессе анализа технического задания и документации были выполнены этапы:

Изучение ТЗ: Определена бизнес-цель — предоставить студентам возможность загружать сканы достижений в личный кабинет.

Анализ взаимодействия: Выявлена необходимость интеграции нового модуля с:

1. «Базой данных студентов» — для верификации данных.
2. «Модулем уведомлений» — для оповещения о новых документах.
3. Анализ API: Изучены эндпоинты:
    1. GET /api/students/{id} для получения данных о студенте.
    2. POST /api/notifications для отправки уведомлений.
4. Определение протоколов: Установлено, что обмен данными происходит по HTTP/1.1 в формате JSON с аутентификацией через JWT-токен.
5. Эндпоинт POST /api/documents/upload для загрузки файлов.
6. Валидация studentId, типа и размера файла (PDF, JPEG, PNG, до 10 МБ).
7. Автоматическая отправка уведомления после успешной загрузки.
8. Безопасность: Использование JWT-токена в заголовках запросов.

9. Надежность: Реализация логики повторных попыток при недоступности сервиса уведомлений.
10. Производительность: Время обработки запроса ≤ 3 секунд.
11. Масштабируемость: Возможность горизонтального масштабирования.

2.2 Выполнение интеграции модулей в программное обеспечение

Для интеграции нового модуля «Загрузка документов» с существующей системой был применен паттерн интеграции на уровне API (API-based Integration). Это позволило обеспечить слабую связанность (loose coupling) компонентов системы.

Этапы интеграции:

- Разработка API-клиентов: На C# созданы клиенты StudentServiceClient и NotificationServiceClient с использованием HttpClient и IHttpClientFactory.
- Реализация аутентификации: Создан сервис ITokenService для управления JWT-токенами, которые автоматически добавляются в заголовки запросов.
- Внедрение логики в основной сервис: В сервис DocumentUploadService через Dependency Injection добавлены клиенты для внешних API. После сохранения файла вызывается метод отправки уведомления.

```
await _notificationServiceClient.SendNotificationAsync(
    new NotificationRequest
    {
        StudentId = uploadRequest.StudentId,
        DocumentType = uploadRequest.DocumentType,
        Status = "pending_verification"
    });
```

Рисунок 1: Пример ключевого фрагмента кода на C#

2.3 Выполнение отладки программного модуля с использованием специализированных программных средств

Для поиска и устранения ошибок применялся комплекс инструментов:

Visual Studio 2022:

1. Визуальный отладчик: Точки останова и пошаговое выполнение для проверки переменных и данных запросов.
2. Immediate Window: Для быстрой проверки выражений и методов.
3. Логирование (ILogger): Добавлены информационные сообщения и ошибки в блоки catch. Анализ логов помог выявить и исправить ошибку аутентификации (статус 401).
4. Postman: Использован для тестирования API-эндпоинтов, что позволило проверить всю цепочку интеграции до разработки клиентского интерфейса.

2.4 Осуществление разработки тестовых наборов и тестовых сценариев

Для модуля «Загрузка документов» была разработана комплексная система тестирования, включающая модульные и интеграционные тесты.

Модульные тесты (Unit Tests): Цель — проверить изолированную логику работы методов, без зависимостей от внешних сервисов. Использовались фреймворки xUnit и Moq для создания mock-объектов.

```
public class DocumentServiceUnitTests
{
    [Fact]
    public void ValidateFile_WithValidJpegFile_ReturnsTrue()
    {
        // Arrange: Подготовка данных
        var fileMock = new Mock<IFormFile>();
        fileMock.Setup(f => f.Length).Returns(1024); // 1 KB
        fileMock.Setup(f => f.ContentType).Returns("image/jpeg");
        var service = new DocumentService();

        // Act: Вызов тестируемого метода
        var result = service.ValidateFile(fileMock.Object);

        // Assert: Проверка результата
        Assert.True(result);
    }

    [Theory]
    [InlineData(15 * 1024 * 1024, "image/jpeg")] // Файл слишком большой
    [InlineData(1024, "application/pdf")]        // Неподдерживаемый тип
    public void ValidateFile_WithInvalidFile_ReturnsFalse(long fileSize, string contentType)
    {
        // Arrange
        var fileMock = new Mock<IFormFile>();
        fileMock.Setup(f => f.Length).Returns(fileSize);
        fileMock.Setup(f => f.ContentType).Returns(contentType);
        var service = new DocumentService();

        // Act
        var result = service.ValidateFile(fileMock.Object);

        // Assert
        Assert.False(result);
    }
}
```

Рисунок 2

Интеграционные тесты (Integration Tests): Цель — проверить взаимодействие модуля с реальными или тестовыми

версиями внешних сервисов (БД, API уведомлений). Для этого использовался WebApplicationFactory из ASP.NET Core.

2.5 Инспектирование компонентов программного обеспечения на предмет соответствия стандартам кодирования

Перед слиянием с основной веткой код прошел обязательное Code Review через Pull Request (MR) в GitLab.

Критерии проверки:

- Соблюдение стиля кодирования C# (именование, отступы).
- Отсутствие «запахов кода» (дублирование, нарушение SRP).
- Наличие комментариев для сложной логики.
- Эффективность и безопасность (отсутствие уязвимостей).
- Качество и покрытие тестами.

Результат ревью: Были получены два основных замечания:

- Строка подключения была «захардкожена» в коде. Исправление: вынесена в appsettings.json.
- В блоке catch не детализировалась ошибка. Исправление: добавлено логирование исключения ex.

3 ВЫПОЛНЕННЫЕ ЗАДАНИЯ

Цель задания: Разработать на C# консольную утилиту для горизонтального объединения двух изображений в одно с соблюдением пропорций и последующим сохранением результата.

Этапы реализации:

- Технологический стек: В качестве основы выбран .NET 6 и библиотека System.Drawing.Common для обеспечения кроссплатформенности и работы с изображениями.
- Загрузка: Исходные изображения загружаются в объекты Bitmap.
- Определение размеров: Высота итогового изображения принимается равной наименьшей высоте из двух исходных для устранения пустых областей и выравнивания.
- Масштабирование: Ширина каждого изображения пересчитывается по формуле (Исходная_Ширина / Исходная_Высота) * Новая_Высота для сохранения пропорций.
- Создание холста: Создается новый Bitmap, шириной равный сумме новых ширин обоих изображений.
- Отрисовка: С помощью графического контекста Graphics оба масштабированных изображения последовательно рисуются на целевом bitmap.
- Сохранение: Результат экспортируется в файл формата JPEG.

```
using System.Drawing;
using System.Drawing.Imaging;

public static class ImageStitcher
{
    public static void StitchImages(string imagePath1, string imagePath2, string outputPath)
    {
        // Загрузка исходных изображений
        using var bitmap1 = new Bitmap(imagePath1);
        using var bitmap2 = new Bitmap(imagePath2);

        // Определение новой высоты как минимальной из двух
        int newHeight = Math.Min(bitmap1.Height, bitmap2.Height);

        // Масштабирование ширины с сохранением пропорций
        int newWidth1 = (int)((float)bitmap1.Width / bitmap1.Height * newHeight);
        int newWidth2 = (int)((float)bitmap2.Width / bitmap2.Height * newHeight);

        // Создание результирующего bitmap
        using var resultBitmap = new Bitmap(newWidth1 + newWidth2, newHeight);
        using var graphics = Graphics.FromImage(resultBitmap);

        // Настройка качества отрисовки
        graphics.InterpolationMode = System.Drawing.Drawing2D.InterpolationMode.HighQualityBicubi
        graphics.SmoothingMode = System.Drawing.Drawing2D.SmoothingMode.HighQuality;

        // Отрисовка изображений на холсте
        graphics.DrawImage(bitmap1, 0, 0, newWidth1, newHeight);
        graphics.DrawImage(bitmap2, newWidth1, 0, newWidth2, newHeight);

        // Сохранение результата с настройками качества JPEG
        var jpegEncoder = GetEncoder(ImageFormat.Jpeg);
        var encoderParameters = new EncoderParameters(1);
        encoderParameters.Param[0] = new EncoderParameter(Encoder.Quality, 90L);

        resultBitmap.Save(outputPath, jpegEncoder, encoderParameters);
    }

    private static ImageCodecInfo GetEncoder(ImageFormat format)
    {
        ImageCodecInfo[] codecs = ImageCodecInfo.GetImageEncoders();
        return codecs.FirstOrDefault(codec => codec.FormatID == format.Guid);
    }
}
```

Рисунок 3

3.2 Задание 2: Разработка тестовой документации для программного модуля

1. Соответствие требованиям: Модуль полностью соответствует всем поставленным функциональным требованиям.
2. Качество алгоритма: Алгоритмы изменения размеров и склейки изображений работают корректно, без визуальных артефактов и искажений.
3. Качество кода: Исходный код написан с соблюдением стандартов C#, отличается высокой читаемостью, четкой структурой и комплексной обработкой исключительных ситуаций.
4. Тестовое покрытие: Достигнуто 100% покрытие тестами, включая основные пользовательские сценарии, граничные условия и обработку ошибок.
5. Готовность: Модуль готов к промышленному использованию и интеграции в системы для пакетной обработки изображений.

Выводы по заданию:

Разработанный программный модуль успешно прошел все этапы тестирования и полностью соответствует техническому заданию. Алгоритм обработки изображений, включая коррекцию размеров и склейку, демонстрирует стабильную работу без визуальных артефактов. Кодовая база отличается высоким качеством: она соответствует стандартам C#, имеет четкую модульную структуру, снабжена комментариями и комплексной обработкой исключений. Полное тестовое покрытие (100%) всех ключевых, граничных и ошибочных сценариев гарантирует надежность решения. Модуль готов к промышленной эксплуатации и может быть интегрирован в

существующие системы для автоматизации пакетной обработки изображений.

Итоги производственной практики:

За период практики с 17 по 30 ноября 2025 года в филиале Университета «Синергия» в г. Череповец были не только выполнены все поставленные задачи, но и достигнуты значительные результаты в профессиональном развитии.

Ключевые результаты и приобретенные компетенции:

Теоретическая подготовка: Была глубоко изучена организационная структура и IT-процессы крупного образовательного учреждения, а также внутренняя регламентирующая документация.

Практический опыт:

- Проектирование: Освоен полный цикл работы с требованиями: от анализа ТЗ до формулирования конкретных критериев к программному обеспечению.
- Backend-разработка: Приобретены углубленные навыки интеграции веб-сервисов через REST API с использованием современных практик (HTTPClient, аутентификация, устойчивость к сбоям через Retry Pattern).
- Инструменты: Освоен эффективный стек технологий для отладки, логирования и тестирования (Visual Studio Debugger, xUnit, Moq).
- Разработка: Успешно реализован и протестирован специализированный модуль для работы с изображениями на C#.

- Качество кода: Получен практический опыт участия в код-ревью, что повысило внимание к чистоте, читаемости и соответствию кода стандартам.
- Командная работа: Интегрирован в рабочий процесс по методологии Agile с использованием Jira, Confluence и GitLab.

ЗАКЛЮЧЕНИЕ

Практика дала бесценный опыт реальной разработки, позволила применить теорию на практике и дала четкое понимание того, как устроена работа в IT. Это мощный толчок для моего дальнейшего профессионального роста.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Университет «Синергия». Официальный сайт. – [Электронный ресурс]. – URL: https://synergy.ru/
2. Microsoft Documentation. C# Guide. – [Электронный ресурс]. – URL: https://docs.microsoft.com/ru-ru/dotnet/csharp/
3. Microsoft Documentation. ASP.NET Core Documentation. – [Электронный ресурс]. – URL: https://docs.microsoft.com/ru-ru/aspnet/core/
4. Microsoft Documentation. System.Drawing.Common Namespace. – [Электронный ресурс]. – URL: https://docs.microsoft.com/ru-ru/dotnet/api/system.drawing
5. xUnit.net Documentation. – [Электронный ресурс]. – URL: https://xunit.net/
6. Moq Documentation. – [Электронный ресурс]. – URL: https://github.com/moq/moq4
7. C# Coding Conventions (Microsoft .NET Guidelines). – [Электронный ресурс]. – URL: https://docs.microsoft.com/ru-ru/dotnet/csharp/fundamentals/coding-style/coding-conventions
8. Martin, R. C. Clean Code: A Handbook of Agile Software Craftsmanship. – Prentice Hall, 2008.
9. Fowler, M. Patterns of Enterprise Application Architecture. – Addison-Wesley, 2002.
10. Внутренняя документация, регламенты и технические задания филиала Университета «Синергия» в г. Череповец (2025 г.).